

บทที่ 4

MPI

MPI หรือ Message Passing Interface เป็นมาตรฐานสำหรับการติดต่อแบบส่งข้อความ (Message Passing) นิยมใช้มากในการคำนวณแบบขนาน (Parallel Computing) และเวนเดอร์ (Vendor) ทั่วไป MPI เป็นมาตรฐานที่ไม่ขึ้นกับแพลตฟอร์ม (Platform-Independent) และไม่ขึ้นกับภาษาที่ใช้เขียน (Language-Independent) สำหรับไลบรารีแบบการส่งข้อความ (Message-Passing Library) โดยได้รับการพัฒนามาจาก MPI Forum ซึ่งเป็นการรวมกันต่างๆระหว่างผู้ขายคอมพิวเตอร์แบบขนาน ผู้เขียนไลบรารี และผู้เชี่ยวชาญด้วยแอปพลิเคชัน (Application Specialist) และไลบรารีของตัว MPI นี้จะใช้ภาษาฟอร์แทรนและซี เป็นหลัก

โมเดลของ Message Passing นี้คือ

- โปรแกรมแบบขนานที่ประกอบไปด้วยชุดของโพรเซส (Set of Processes) ซึ่งโพรเซสเหล่านี้จะถูกจับคู่ (Map) กับโพรเซสหนึ่งหรือหลายๆโพรเซส
- การคำนวณที่ซับซ้อนจะถูกแบ่งออกเป็นโพรย่อยๆเหล่านั้นและจะถูกคำนวณไปด้วยกัน (Concurrently)
- การติดต่อระหว่างโพรเซสจะใช้การส่งข้อความ ทั้งข้อมูล (Data) และ Synchronization

MPI ได้ถูกกำหนดไว้โดย MPI Forum ซึ่งมีวิวัฒนาการคือ

- MPI 1.1 ซึ่งถูกพัฒนาจาก 1992 – 1994
- MPI 2.0 พัฒนาระหว่าง 1995 – 1997

ซึ่งได้มีหลายหน่วยงานที่เป็นคนนำไปพัฒนา (Implement) อยู่หลายฝ่ายเช่น

- MPICH : Argonne National Laboratory
- LAM : Notre-Dam University
- MPI-PRO : MPITech

สาเหตุที่ทำให้ MPI นิยมใช้กันนั้นเป็นเพราะว่ารายละเอียดของมันไม่ขึ้นกับเวนเดอร์ ระบบปฏิบัติการ ฮาร์ดแวร์ และภาษาที่ใช้ ซึ่งหมายความว่า โปรแกรมใดก็ตามที่เขียนโดยมีพื้นฐานมาจาก MPI (Based on MPI) นั้นสามารถที่จะคอมไพล์ (Compile) และรัน (Run) ได้โดยไม่ต้องเปลี่ยนซอร์สโค้ด (Source Code) บนพีซี เวอร์คสเตชัน SMP COW MPP และอื่นๆ จากเวนเดอร์หรือระบบปฏิบัติการใดๆ

นอกจาก MPI แล้วยังมีไลบรารีที่สำคัญคือ PVM ซึ่งเป็นโปรแกรมแบบขนานเช่นเดียวกัน โดยได้มีการพัฒนามาก่อนไลบรารี MPI แต่ปัจจุบันนี้คนนิยมใช้ไลบรารีของ MPI มากกว่า เนื่องจากมีฟังก์ชันการทำงานที่มากกว่า PVM และใช้ได้ง่ายกว่าด้วย

- สำหรับ MPI นั้น กลุ่มของโพรเซส (Process Group) จะถูกแยกแยะได้ด้วย MPI_COMM_WORLD และไม่มีโพรเซสใดที่สามารถจะถูกสร้างขึ้นหรือถูกทำลายลงในระหว่างการรันของโปรแกรม
- MPI ไม่ใช่สแตนด์อโลน (Standalone) หรือไม่ใช่ระบบซอฟต์แวร์ที่ Self-Contained มันเป็นส่วนหนึ่งของ Message-Passing (Message-Passing Layer) ที่อยู่บนสุดของโปรแกรม (Native Programming Environment) ดังนั้นเราไม่จำเป็นต้องเรียนรู้การเขียน UNIX Programming Environment ด้วย

4.1 องค์ประกอบต่างๆของ MPI

4.1.1 MPI โพรเซส

MPI โพรเซสเป็นส่วนหนึ่งของเอนทิตี (Entity) ที่ทำงานเป็นคอมพิวเตอร์ทาสก์ (Computer Task) ถ้าพิจารณา MPI ในรูปแบบโปรแกรมมิ่งโมเดล แล้วจะเห็นว่าแต่ละโพรเซสจะเชื่อมโยงกันโดยแต่ละโพรเซสจะมีหน่วยความจำของตัวเองเรียกว่า หน่วยความจำโลคอล (Local Memory) ในโมเดลจะให้โพรเซสเชื่อมโยงกันด้วยลักษณะหน่วยความจำที่คล้ายๆ กันเป็นการทำงานลักษณะ Multiple Processes Multiple Data (MIMD)

โพรเซสใน MPI จะสื่อสารกันด้วยข้อความ (Message) ซึ่งข้อความจะประกอบไปด้วยส่วนของข้อมูลเรียกว่า Body ประกับข้อมูลที่เพิ่มเข้าไปบางข้อมูลเรียกว่าข้อมูลหน้าซอง (Envelop) โดยข้อมูลหน้าซองจะเป็นส่วนที่ชี้เฉพาะให้โพรเซสทราบถึงความแตกต่างกันระหว่างแต่ละข้อความ และผู้รับสามารถตัดสินใจและเลือกรับข้อมูลได้อย่างถูกต้อง

การแลกเปลี่ยนข้อความระหว่างโพรเซสใน MPI จะหมายถึงการสื่อสาร (Communication) กล่าวคือ MPI โพรเซสจะรับข้อมูลไปประมวลผลในหน่วยความจำโลคอลหรือทำการซิงโครไนส์กับโพรเซสอื่น โดยการสื่อสารจะส่งการเรียกในรูปฟังก์ชันการทำงานของภาษาซี หรือฟอร์แทรน

4.1.2 กลุ่มโพรเซส (Process Group)

กลุ่มโพรเซสใน MPI เป็นเซตของโพรเซสซึ่งเป็นกลุ่มอยู่และบ่งบอกถึงแต่ละโพรเซสด้วยเลขเรียงกันไปตั้งแต่ 0 เป็นต้นไป โดยจำนวนของโพรเซสที่ต่อเชื่อมกันในกลุ่มจะเรียกว่า อันดับ (Rank) ของกลุ่ม ลักษณะการทำงานของโพรเซสในกลุ่มจะทำงานร่วมกัน ในแต่ละงานจะมีโพรเซสที่ทำงานอยู่เป็น

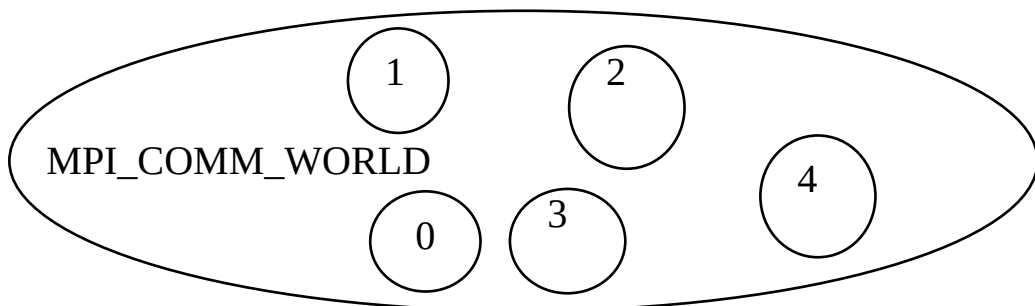
จำนวนแน่นอน และไม่สามารถเพิ่มจำนวนของโพรเซสระหว่างทำการรันโปรแกรมได้ แต่ MPI สามารถขยายอันดับของกลุ่มได้

4.1.3 คอมมิวนิเคชัน คอนเท็ก

ในแต่ละโพรเซส MPI จะสามารถตัดสินใจเลือกที่จะรับหรือไม่รับข้อความได้ขึ้นอยู่กับข้อมูลหน้าของของข้อความนั้นๆ ซึ่งข้อผิดพลาดทั่วไปของการส่งข้อความ คือ โพรเซสได้รับข้อความที่ส่งมาโดยผิดโพรเซส ในจุดนี้เราสามารถกำหนดค่าโดยตรงให้โดยผู้ใช้ หรือจะใช้การเรียกใช้คอนเท็กใน Message Passing Interface จะสนับสนุน ที่จะมาช่วยให้การส่งข้อความไม่เกิดความเลียงและความสับสนขึ้น

4.1.4 ตัวติดต่อสื่อสาร (Communicator)

ตัวติดต่อสื่อสารจะถูกสร้างมาเชื่อมโยงแต่ละ MPI โพรเซส ในแต่ละกลุ่มโพรเซส จากแนวคิดของเวอร์ชวล ทอโปโลยี โดยจะทำหน้าที่จัดการกับการสื่อสารทุกๆ การสื่อสารของ MPI ต้องเรียกผ่านตัวติดต่อสื่อสารและโพรเซส ที่จะทำการสื่อสารกันได้นั้นต้องมีการแชร์ตัวกันเสียก่อน ซึ่งในแต่ละตัวติดต่อสื่อสารจะประกอบด้วยลิสต์ของโพรเซสในกลุ่ม โดยปกติแล้วทุกๆ โพรเซสจะเป็นสมาชิกของตัวติดต่อสื่อสาร MPI_COMM_WORLD



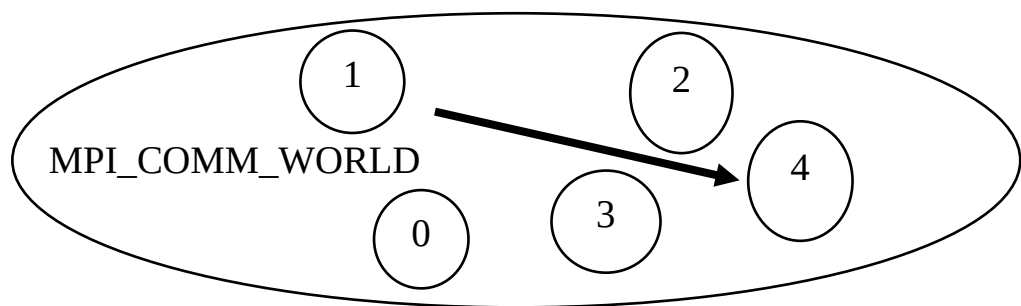
รูปภาพที่ 4-1 แสดงตัวติดต่อสื่อสาร (Communicator)

4.2 การติดต่อสื่อสารของ MPI (MPI Communication)

การทำงานของ MPI จะอยู่บนพื้นฐานบนการสื่อสารกันระหว่างโพรเซส โดยการใช้การส่งข้อความไปทำการประมวลผลในโหนดต่างๆ มีลักษณะการสื่อสารกันหลายลักษณะเป็นบล็อกกิ้ง เวอร์ชันและนอน-บล็อกกิ้ง เวอร์ชัน ในรูปแบบบล็อกกิ้ง เวอร์ชัน จะมีการสร้างรูทีนในการรับส่งขึ้น และทำการส่งค่ากลับจากรูทีน เมื่อเสร็จสิ้นแล้ว ส่วนในแบบนอน-บล็อกกิ้ง เวอร์ชันจะมีการสร้างรูทีนขึ้นมาเช่นกันแต่จะมีการส่งค่ากลับมาก่อนที่จะทำการสื่อสารจะเสร็จสิ้น โดยโพรเซสที่ทำการส่งและรับจะมีการสร้างบัฟเฟอร์ขึ้นก่อนที่จะโพรเซสผู้ส่งจะทำการอัปเดต บัฟเฟอร์ได้ต้องตรวจสอบรูทีน ก่อนว่าการสื่อสารเสร็จสมบูรณ์หรือยัง โดยสามารถเรียกดูที่ทำการทดสอบได้ ส่วนโพรเซสผู้รับจะสามารถทำงานอื่น หรือประมวลผลอย่างอื่นก่อนได้จนกว่าจะถึงเวลาที่ต้องการใช้ข้อมูลก็จะไปตรวจสอบจากบัฟเฟอร์ว่ามีข้อมูลอยู่หรือไม่

4.2.1 การสื่อสารแบบจุดต่อจุด Point-to-Point Communication

เป็นการสื่อสารที่มีโพรเซสเข้ามาเกี่ยวข้องกัน 2 โพรเซสเท่านั้นโดยโพรเซสหนึ่งจะส่งข้อความ และอีกโพรเซสหนึ่งจะเป็นผู้รับข้อความ โดยตัวส่งจะใช้การเรียก MPI Call ไปยังปลายทางในอันดับของตัวติดต่อสื่อสารที่เหมาะสม สำหรับข้อมูลที่ส่งในตัวของการสื่อสารจะประกอบไปด้วย แอดเดรสเริ่มต้นของข้อมูล ขนาดข้อมูล ประเภทข้อมูล จุดหมาย และอาร์กิวเมนต์ ของตัวติดต่อสื่อสาร หลักการในการส่งจะเรียกว่า Pushing คือการส่งข้อความไปยังโพรเซสต่างๆโดยตัวโพรเซสไม่สามารถ Fetch ข้อมูลเองได้ ทำได้แต่เพียงรอรับข้อมูลที่ส่งมาแล้วเท่านั้น โดยในแบบจุดต่อจุดจะสามารถทำการสื่อสารได้ 4 รูปแบบ



รูปภาพที่ 4-2 การสื่อสารแบบจุดต่อจุด

4.2.1.1 การส่งแบบซิงโครนัส (Synchronous Send)

การส่งข้อความในแบบนี้จะต้องรู้ว่าผู้รับได้รับข้อความแล้ว โดยตัวโพรเซสผู้รับจะส่งสัญญาณตอบกลับมากเพื่อบอกว่าการสื่อสารสมบูรณ์แล้วการส่งในโมดนี้จะช่วยให้ทำการดีบั๊กได้ง่ายและจะไม่ทำให้เกิดการโอเวอร์โหลดและส่งข้อความผิดได้ เนื่องจากต้องมีการซิงโครไนส์กันระหว่างผู้ส่งกับผู้รับ และถ้าใช้ร่วมกับ บล็อกกิ้ง จะทำให้เกิดปัญหาในลักษณะเดดล็อกได้ คือผู้ส่งและผู้รับจะมีการซิงโครไนส์ต่อกันไปเรื่อยๆ ทำให้มีการรอข้อความในแต่ละโพรเซส สามารถแก้ไขได้โดยใช้นอน-บล็อกกิ้งแทน

4.2.1.2 การส่งแบบใช้บัฟเฟอร์ (Buffer Send)

จะมีการรับประกันการส่งข้อความทันทีที่ทำการส่ง โดยจะใช้วิธีการทำสำเนาข้อความไว้ในระบบบัฟเฟอร์หลังจากทำการส่ง วิธีนี้จะรับประกันการทำงานโดยไม่ต้องมีการซิงโครไนส์กันระหว่างโพรเซสทำให้สามารถทำงานได้เร็วและสะดวกขึ้น แต่จะสามารถเกิดการโอเวอร์โหลดของเครือข่ายขึ้นได้จากการส่งมากๆ ข้อเสียของการส่งแบบบัฟเฟอร์คือโปรแกรมเมอร์จะไม่สามารถกำหนดขนาดบัฟเฟอร์ได้แน่นอนและจะต้องมีบัฟเฟอร์ที่มีขนาดเพียงพอกับโปรแกรมอีกด้วย

4.2.1.3 การส่งแบบพร้อมทำงาน (Ready Send)

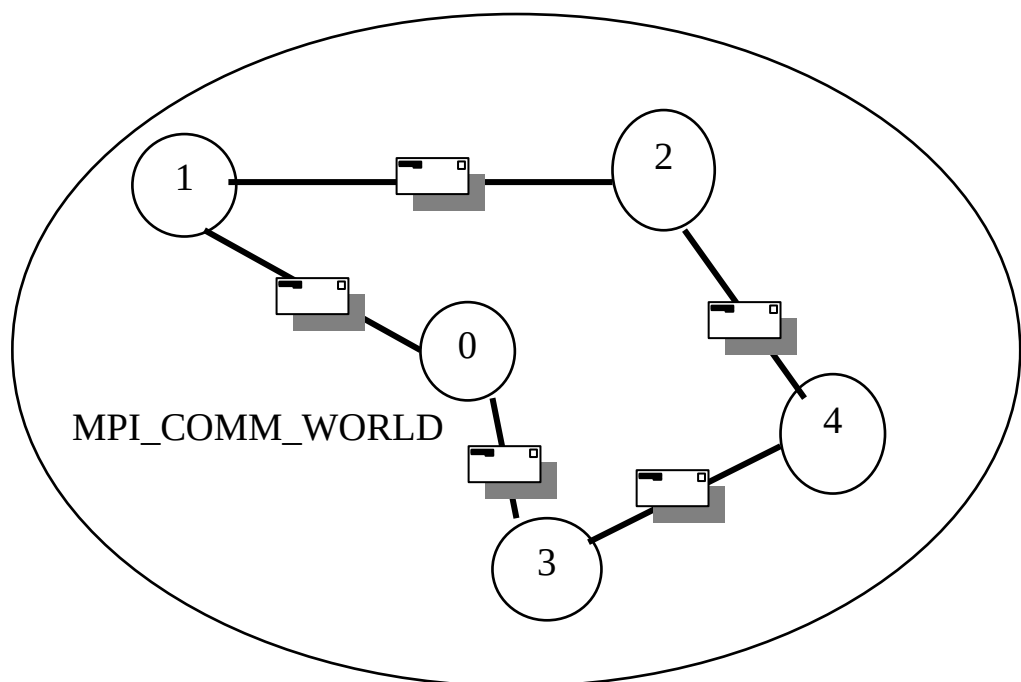
มีลักษณะคล้ายกับการส่งแบบบัฟเฟอร์ คือจะรับประกันการส่งข้อความทันที การสื่อสารแบบนี้ จะรับประกันความถูกต้อง ถ้ามีผู้รับที่ถูกต้องมารอรับ โดยโปรเซสผู้ส่งจะส่งข้อความไปยังเครือข่ายการสื่อสารและหวังว่าผู้รับจะรออยู่เพื่อรับข้อความ ถ้าโปรเซสผู้รับพร้อมที่จะรับข้อความ ก็จะทำการรับแต่ถ้าไม่เช่นนั้น ข้อมูลจะเจียบหายไปและเกิดความผิดพลาด

แนวคิดนี้จะทำการหลีกเลี่ยงการทำแฮนเชกและบัฟเฟอร์รีจระหว่างผู้ส่งและผู้รับ ในโหมดนี้ยาก ในการทำดีบั๊ก และต้องการการดูแลในการเขียนโปรแกรม

4.2.1.4 การส่งแบบมาตรฐาน (Standard Send)

โหมดนี้จะสามารถเลือกใช้ระหว่างซิงโครนัสและบัฟเฟอร์ คือการเสร็จการส่งอาจจะก่อนที่จะมีการตรวจรับจากผู้รับ หรือไม่ก็ได้ โดยในการส่งแบบมาตรฐานจะมีหลักว่า

- ไม่ควรตั้งสมมติฐานว่าการส่งข้อความเสร็จก่อนที่ผู้รับจะเริ่มทำการรับ เพราะถ้าใช้การบล็อกก็อาจเป็นสาเหตุให้เกิดเดดล็อกได้



รูปภาพที่ 4-3 แสดงการเกิดเดดล็อก

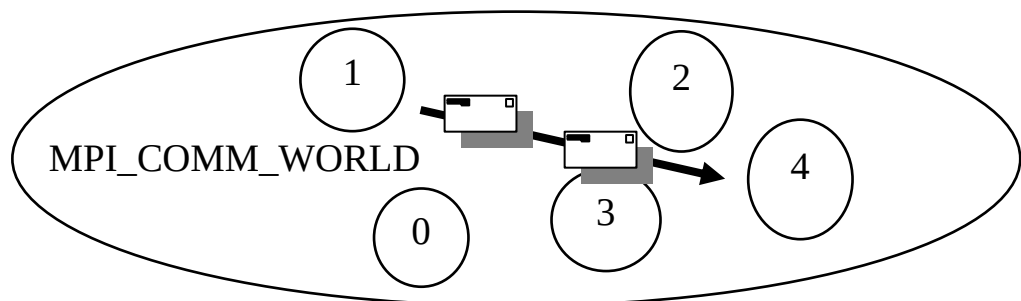
- ไม่ควรตั้งสมมติฐานว่าการส่งเสร็จสิ้นเมื่อผู้รับเริ่มทำการรับ
กล่าวโดยสรุปคือการส่งแบบมาตรฐานอาจจะนำวิธีซิงโครนัสหรือวิธีบัฟเฟอร์มาใช้ก็ได้และผู้ใช้ไม่ควรตั้งสมมติฐานทั้ง 2 กรณี เพราะไม่สามารถรู้ได้

การรับส่งข้อมูลผู้รับจะพิจารณาจากข้อมูลหน้าซองซึ่งเป็นส่วนที่แสดงความแตกต่างระหว่างข้อความและในส่วนแทกบนข้อความจะเป็นตัวบ่งบอกถึงประเภทของข้อความต่างๆกัน เช่น ข้อมูลเริ่มต้น (Initial Data) ลักษณะการร้องขอจากโพรเซสอื่นๆ ในส่วนแทกนี้จะทำให้ผู้รับสามารถเลือกข้อความที่ต้องการรับได้อย่างถูกต้อง

4.2.2 การรับประกันการสื่อสารแบบจุดต่อจุด

4.2.2.1 ความถูกต้องของลำดับการส่ง (Message Order Prevention)

ข้อความจะต้องไม่มีการแซงกันในการส่ง ต้องเป็นไปตามลำดับ เช่น โพรเซส A ส่งสองข้อความไปยังโพรเซส B แล้วโพรเซส B ทำการโพส เพื่อรับทั้งสองข้อความต้องมีการรับประกันว่าการรับจะเป็นไปตามลำดับที่ทำการส่งมา



รูปภาพที่ 4-4 รับประกันความถูกต้องของการส่ง

4.2.2.2 ความคืบหน้าในการส่ง (Progress)

จะไม่มีกรณีการหลุดโพรเซสในการรับส่งนอกจากสองโพรเซสที่เกี่ยวข้องกันเท่านั้น การรับส่งจะเสร็จสมบูรณ์อย่างถูกต้องเมื่อโพรเซสที่ 1 ส่งข้อความไปยังโพรเซสที่ 2 ที่มีคุณสมบัติตรงกับข้อความที่ส่งไป แต่อาจมี 2 ลักษณะที่เป็นไปได้และไม่ควรเกิดขึ้น

- ข้อความที่ส่งถูกรับไปโดยโพรเซสที่ 3 และตรงกับการรับข้อความที่ส่งมาทำให้การส่งสมบูรณ์ แต่โพรเซสที่ 2 ไม่ได้รับข้อความ
- ข้อความโพรเซสที่ 3 ส่งข้อความออกไป และได้รับโดยโพรเซสที่ 2 ซึ่งการสื่อสารก็จะสมบูรณ์ แต่โพรเซสที่ 1 ไม่ได้ทำการส่ง

4.2.3 การสื่อสารแบบกลุ่ม (Collective Communication)

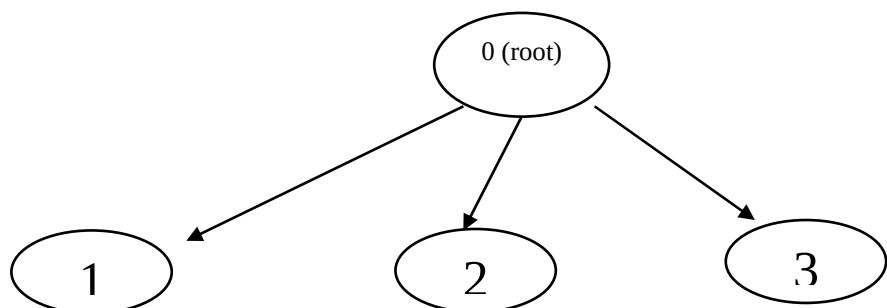
จะเป็นการสื่อสารที่ทุกๆ โพรเซสภายในตัวติดต่อสื่อสาร (คือทุกๆโพรเซสในกลุ่ม ที่เชื่อมโยงกันกับตัวติดต่อสื่อสาร) และถ้าหากมีการใช้การสื่อสารแบบกลุ่มในส่วนสับเซตของตัวติดต่อสื่อสารจะต้องมีการสร้างตัวติดต่อสื่อสารใหม่ขึ้น

4.2.3.1 คุณลักษณะของการสื่อสารแบบกลุ่ม

- การสื่อสารแบบกลุ่มจะไม่มี การเข้าไปเกี่ยวข้องกับการสื่อสารแบบจุดต่อจุด โดยจะแยกส่วนกันอย่างชัดเจน ตัวอย่างเช่น การสื่อสารแบบกลุ่มจะไม่สามารถส่งข้อความและรับโดยจะไม่สามารถส่งข้อความและรับโดยผู้รับที่ใช้การสื่อสารแบบจุดต่อจุดได้ แต่ใน ตัวติดต่อสื่อสารหนึ่งที่มี 2 ตัวติดต่อสื่อสารย่อยทั้ง 2 ตัวจะสามารถแยก เป็นการสื่อสารแบบจุดต่อจุดกับการสื่อสารแบบกลุ่มได้
- การสื่อสารแบบกลุ่มจะใช้การชิงโครน์หรือไม่ได้
- ในการสื่อสารแบบกลุ่มจะมีลักษณะเป็นบล็อกกิ้ง เวอร์ชันได้เท่านั้น
- ทุกๆ โพรเซสในตัวติดต่อสื่อสารต้องเรียกใช้การสื่อสารแบบกลุ่มลักษณะที่เหมือนกันกับการสื่อสารแบบจุดต่อจุด
- ข้อความเป็นประเภทข้อมูลของอาร์เรย์ ชนิดหนึ่งๆ
- ประเภทข้อมูลจะต้องมีคุณสมบัติตรงกันระหว่างผู้ส่งและผู้รับ ลักษณะที่ต่างกับการสื่อสารแบบจุดต่อจุด
- ไม่มีหลักการเกี่ยวกับแท็กของข้อความ
- ข้อความที่ถูกส่งมาจะต้องถูกใส่ลงในบัฟเฟอร์ของผู้รับนั้นๆ

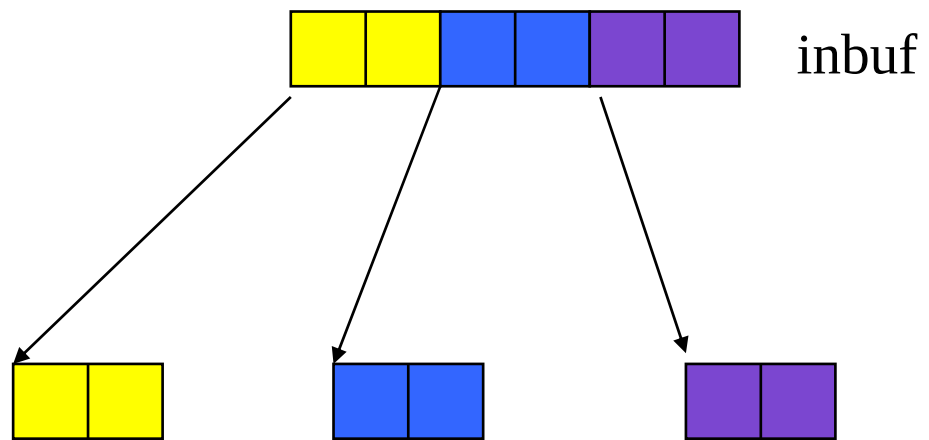
4.2.3.2 ลักษณะการส่งในแบบกลุ่ม

- **Broadcast** : จะทำการกระจายข้อความที่เหมือนกันจากโพรเซสหนึ่งที่เรียกว่า โพรเซสรูทไปยังโพรเซสอื่นๆ ที่เชื่อมต่อกับตัวติดต่อสื่อสาร แบบอื่นๆจะมีลักษณะตรงกันข้าม



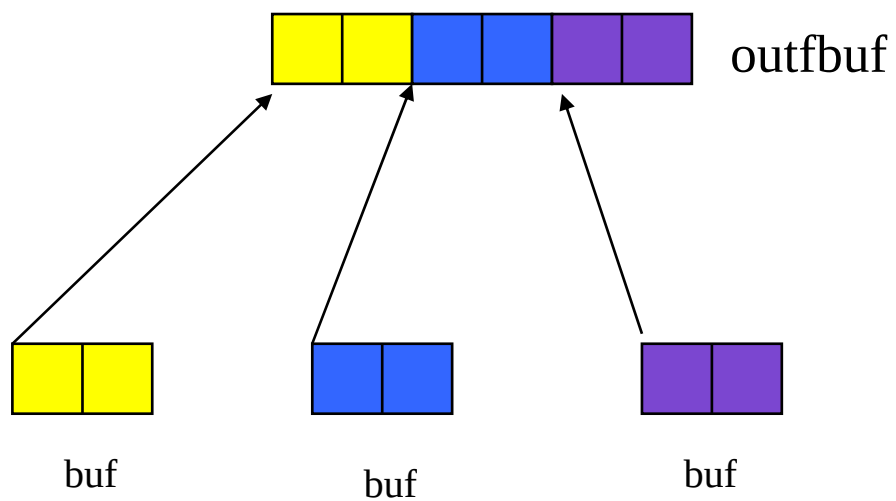
รูปภาพที่ 4-5 การส่งแบบกลุ่ม Broadcast

- **Scatter** : โพรเซสเซอร์จะส่งข้อความที่ต่างกันที่มีลักษณะของข้อมูล เหมือนกันไปยังแต่ละโพรเซสเซอร์



รูปภาพที่ 4-6 การส่งแบบกลุ่ม Scatter

- **Gather** : โพรเซสเซอร์จะรับข้อความที่ต่างกันจากแต่ละโพรเซสเซอร์ มาทำการต่อกันเป็นข้อความในรูปประเภทข้อมูลเดียวกัน



รูปภาพที่ 4-7 การส่งแบบกลุ่ม Gather

- **Reduce , Scan** : ทุกๆ อีลิเมนต์ในตำแหน่งเดียวกันจะเป็นอินพุตจากทุกๆ โพรเซสเซอร์ที่จะทำการรีดิวซ์มารวมกัน โดยผลลัพธ์จากการรีดิวซ์จะเก็บเป็นผลลัพธ์ในโพรเซสเซอร์
- **Reduction** : จะทำการรวมหรือเชื่อมต่อข้อความ คล้ายกับ Gather

- **Scan Operation** : จะทำแบ่งส่วน (Patial) ผลลัพธ์ไปเก็บในทุกๆ โพรเซส ในรูปลำดับ บางอย่างโดยการสแกน จะใช้ประโยชน์ในการสร้างค่าลักษณะเฉพาะ (Unique) สำหรับแต่ละโพรเซส
- **Gather-to-all** : จะทำการเกเตอร์ข้อความมาไว้ที่โพรเซสครูท และตามด้วยการทำบรอดคาสต์ ข้อมูลที่ได้จากการเกเตอร์ไปยังทุกโพรเซส

4.3 สิ่งที่เกี่ยวข้องกับการเขียนโปรแกรม MPI

4.3.1 Text Editor

เนื่องจากว่าไมโครโปรแกรมสำเร็จรูปสำหรับเขียนโปรแกรมเช่นพวก Visual C หรือ Borland C เหมือนบน Windows เราสามารถใช้ตัวใดก็ได้เช่น Ex, Pico, Vi และอื่นๆ

4.3.2 Debugging Tools

ใน UNIX นั้นมีพวก gdb และ dbx ให้อยู่แล้ว เราสามารถหา MPI Manual ได้ออนไลน์

`Mpirun -dbg = gdb a.out`

หรือเราจะใช้ printf ก็ได้เช่นกัน ซึ่งเป็นวิธีที่ทำได้ง่ายมาก

ถ้าจะให้ง่ายการดีบั๊กโค้ดควรจจะรันบนเครื่องแก่เครื่องเดียวเท่านั้น

4.3.3 .cshrc

เป็นไฟล์ที่บอกรายละเอียดเกี่ยวกับ Shell เราต้องทำการเพิ่มบรรทัดเพื่อเซตค่าพาธ (Path) ด้วย (\$path/user/local/...) เพื่อดังค่าพาธให้กับชีคมไพล์เลอร์

4.3.4 การลบโพรเซส(Kill Process)

ที่ไม่ต้องการเนื่องจากว่าบางครั้งนั้นโพรเซสที่อยู่บนโนดอื่น ไม่ถูกลบเมื่อโปรแกรมแบบขนานปิดตัวเองลงปกติ (เรียกว่า Segmentation Fault) โพรเซสเหล่านั้นเราจะต้องเป็นคนลบมันเองด้วยคำสั่งดังนี้

`ps -ef | grep userid`

หา Process ID

`kill pid`

ทำการลบโพรเซส

4.3.5 rsh

MPI จะเริ่มต้นการรีโมตโพรเซส (Remote Process) ด้วยการ ใช้ rsh (Remote-Shell) เป็นโปรแกรมที่มีอยู่ใน UNIX OS ถ้าเราทำการรันโปรแกรมและไม่มีคำสั่งผิดพลาด “Permission Denied” หมายความว่า Remote Shell นั้นไม่สามารถใช้ไบนารี่ซอร์สนั้นๆ เราจะต้องทำการตั้งค่า rsh ก่อน

4.3.6 การคอมไพล์โปรแกรมแบบขนาน

- สำหรับโปรแกรมธรรมดาที่ไม่ได้มีการประกาศใช้ User-Defined Library

```
mpicc -c foo.c
mpicc -o foo foo.c
mpicc -o foo foo.c -lm
```
- สำหรับโปรแกรมที่มีการทำงานซับซ้อนกว่านั้น ต้องใช้ makefile และพิมพ์ 'make' เพื่อที่จะคอมไพล์

4.3.7 การรันโปรแกรมแบบขนาน

ใช้คำสั่ง 'mpirun -help' เพื่อที่จะขอคู่มือตัวเลือกต่างๆ เช่น

- mpirun -np 4 a.out เป็นการรัน a.out บนซีพียู 4 ตัวสำหรับเครื่อง SMPs
- mpirun -p4pg pgfile a.out ตัวเลือกนี้ทำให้มีสิทธิ์ทุกอย่างในการบังคับทุกโพรเซสที่รันอยู่ ซึ่งเราสามารถรันโพรเซสต่างกันบนต่างโหนดได้ และทำให้เราสามารถเจาะจงลงไปเลยได้ว่าจะให้โพรเซสเท่าไรบนหนึ่งเครื่อง อย่างเช่น

```
sgi1      0      /home/root/myprog
sgi2      1      /home/root/myprog
sgi3      2      /home/root/myprog
sgi4      3      /home/root/myprog
```

pgfile จะรัน 5 ครั้ง ซึ่งอันนี้รันบน sgi2 อีกสองอันอยู่ที่ sgi3 โดยใช้วิธีการติดต่อแบบหน่วยความจำร่วมกัน และอีกสองอันอยู่บน sgi1 โดยใช้การติดต่อของซ็อกเก็ต

4.4 ฟังก์ชันพื้นฐานของ MPI

MPI นั้นมีฟังก์ชันอยู่มากมาย (129 Function) แต่สำหรับการเริ่มต้นเรียนรู้นั้นฟังก์ชันที่ควรจะศึกษาก่อนมีอยู่ 6 อย่างด้วยกันคือ

4.4.1 MPI_INIT(int *argc, char **argv);

เป็นการเริ่มต้นการใช้ MPI ก่อนรันโปรแกรมแบบ MPI ตัวอย่างคือ

```
int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
}
```

4.4.2 MPI_FINALIZE();

ฟังก์ชันที่เป็นตัวบอกว่าจบการใช้ MPI ตัวอย่างเช่น

```
int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    MPI_Finalize();
}
```

4.4.3 MPI_COMM_Size(MPI_COMM comm, int *size);

มีโปรเซสที่เกี่ยวข้อง ตัวอย่างเช่น

```
int mysize;
MPI_COMM_Size(MPI_COMM_WORLD, &mysize);
```

4.4.4 MPI_COMM_Rank(MPI_COMM comm, int *rank);

เป็นตัวค้นหาอันดับของโปรเซส เช่น

```
int rank;
MPI_COMM_Rank(MPI_COMM_WORLD, &rank);
```

4.4.5 MPI_Send(message, count, datatype, dest, tag, comm);

ฟังก์ชันที่ใช้ในการส่งข้อความ โดยบัฟเฟอร์ข้อความจะถูกเก็บไว้ที่ส่วน (message, count, datatype) และโปรเซสใดที่เป็นปลายทางของฟังก์ชันนี้เก็บไว้ที่ dest ซึ่งเป็นปลายทางของตัวติดต่อสื่อสาร (Communicator) ใน Comm และเมื่อฟังก์ชันนี้คืนค่าหมายความว่าข้อความได้ถูกส่งออกไปแล้ว และบัฟเฟอร์สามารถนำมาใช้ได้ อีก แต่ว่าไม่ได้บอกว่าข้อความได้ถูกรับแล้ว

4.4.6 MPI_Recv(message, count, datatype, source, tag, comm, status);

ฟังก์ชันที่ใช้ในการรับข้อความ ฟังก์ชันจะรอข้อความที่เข้ากับ (source, tag) ถึงจะรับได้ source เป็นอันดับของตัวติดต่อสื่อสารที่อยู่ใน comm. หรือสามารถใช้

MPI_ANY_SOURCE

การที่จะรับข้อความที่น้อยกว่าค่า count นั้นสามารถทำได้ แต่รับมากกว่าไม่ได้จะ error

ค่า status นั้นเป็นข้อมูลเพิ่มเติมที่เป็นโครงสร้างของข้อมูล (Data Structure) ที่เราสามารถประกาศใช้ในโปรแกรมได้เช่น

```
int recvd_tag, recvd_from, recvd_count;

MPI_Status status;

MPI_Recv(..., MPI_ANY_SOURCE, MPI_ANY_TAG, ..., &status);

recvd_tag = status.MPI_TAG;

recvd_from = status.MPI_SOURCE;

MPI_Get_count(&status, datatype, &recvd_count);
```

จะเห็นได้ว่าเราสามารถรับข้อความมาจากไหนก็ได้ (MPI_ANY_SOURCE) และเราก็สามารถจะดึงค่าอันดับออกมาได้ด้วยตัวแปร recvd_from และเช่นเดียวกับตัวแปร recvd_tag เราสามารถรับค่า tag เท่าไหร่ก็ได้แล้วมาดูอีกที

โดยค่า tag นั้นเป็นเหมือนตัวระบุว่าเป็นข้อความอันดับที่เท่าไรซึ่งเราสามารถกำหนดให้กับข้อความเองได้ เพื่อที่จะช่วยโปรเซสที่จะรับข้อความจะได้ระบุได้ว่าควรจะได้รับข้อความไหน ซึ่งทำให้ข้อความนั้นถูกกรอง (ด้วยการกำหนด tag) และไม่ถูกกรอง (ด้วยการใช้ฟังก์ชัน MPI_ANY_TAG เป็นตัวรับทุกๆ tag)

สำหรับชนิดของข้อมูลเปรียบเทียบกับภาษาซีได้ดังนี้

MPI_Datatypes	C data type
MPI_CHAR	char
MPI_SHORT	short
MPI_INT	int
MPI_LONG	long
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_BYTE	

MPI_PACKED	
------------	--

ตารางที่ 4-1 ตารางเปรียบเทียบชนิดข้อมูลของภาษาซีและ MPI

4.5 ตัวอย่างโปรแกรม Hello World

โปรแกรมนี้เป็นโปรแกรมคลาสสิกที่มีอยู่หลายภาษา แต่ละโพรเซสจะส่งเมสเสจว่า ‘Hello World’ มาที่โพรเซส 0 และ โพรเซส 0 จะทำการพิมพ์ข้อความนั้นออกมา

ใน MPI นั้นแต่ละโพรเซสที่เกี่ยวข้องกับการรันนั้นจะถูกระบุไว้ด้วยตัวเลขที่ไม่ติดลบโดยนับไปเรื่อยๆ แต่ละโพรเซสจะมี rank และ id ที่อยู่ระหว่าง 0 ถึง p-1 เมื่อ p คือจำนวนของซีพียูที่เกี่ยวข้องกับการคำนวณนั้น

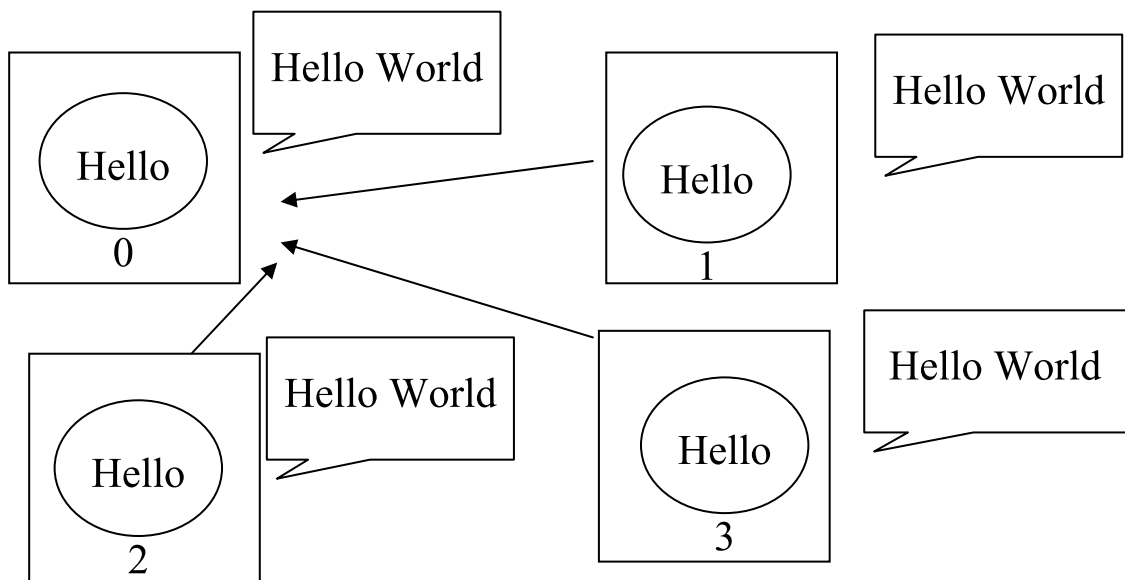
```
main (int argc, char **argv)
{
    int my_rank;
    int p;
    int source;
    int dest;
    int tag=0;
    char message[100];
    MPI_status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_Rank(MPI_COMM_WORLD, &myrank);
    MPI_Comm_Size(MPI_COMM_WORLD, &p);
    If ( myrank != 0)
    {
        sprintf (message, "Hello world from %d", myrank);
        dest=0;
        MPI_send (message, strlen(message)+1, MPI_CHAR, dest, tag,
                  MPI_COMM_WORLD);
    }
}
```

```

else
{
    for (source=1; source<p; source++)
    {
        MPI_Recv(message, 100, MPI_CHAR, source, tag,
                MPI_COMM_WORLD, &status);
        printf("%s \n",message);
    }
}
MPI_FINALIZE();
}

```



รูปภาพที่ 4-8 ตัวอย่างโปรแกรม Hello World

ซึ่งทำการรันแล้วจะได้ผลดังนี้

Hello world from 1

Hello world from 2

Hello world from 3

ซึ่งโปรแกรมนี้เป็นแบบ SPMD (Single Processor Multiple Data) ถึงแม้ว่าโพรเซสที่ 0 จะทำการรันโค้ดที่ต่างไปจากโพรเซสอื่นๆ แต่เราใช้ if-else แทนการใช้โปรแกรมต่างกัน

ซึ่งมีคำสั่งที่เกี่ยวข้องดังนี้

- MPI_Init : เป็นฟังก์ชันที่ต้องทำการเรียกขึ้นมาทุกครั้งก่อนทำการใช้ฟังก์ชันอื่นๆของ MPI
- MPI_Finalize : เป็นฟังก์ชันที่ต้องทำการเรียกขึ้นมาทุกครั้งหลังฟังก์ชันอื่นๆของ MPI เพื่อที่จะได้ล้างเมมโมรี (Clean up memory allocated) ซึ่งถูกเรียกใช้โดย MPI
- Flow Control ในโปรแกรมแบบ SPMD นั้นจะขึ้นอยู่กับอันดับของโพรเซส (Process Rank) ซึ่ง MPI ได้จัดเตรียมฟังก์ชัน MPI_COMM_Rank ไว้แล้วซึ่งมันจะคืนค่าอันดับ (rank) ของแต่ละโพรเซสออกมาให้ ซึ่งพารามิเตอร์ตัวแรกคือตัวจัดการการสื่อสาร ซึ่งจะจัดให้ทุกๆโพรเซสวิ่งเมื่อเริ่มการรันของโปรแกรม
- อีกอย่างที่สำคัญคือจำนวนของซีพียูที่จะถูกเริ่มรัน ซึ่งมันจะถูกประกาศไว้โดย MPI_COMM_Size
- ข้อความที่ถูกส่งไปจริงๆจะกระทำผ่านฟังก์ชัน MPI_Send และ MPI_Recv

MPI_Send (void *message, int count, MPI_datatype datatype, int dest, int tag, MPI_COMM Comm)

MPI_Recv(void *message, int count, MPI_datatype datatype, int source, int tag, MPI_COMM Comm, MPI_status *status)

- ข้อความเป็นข้อมูลที่ถูกเก็บไว้ในบล็อคของเมมโมรี สามารถเป็นข้อมูลชนิดอะไรก็ได้ เราแค่ต้องระบุลงไปถึงพอยน์เตอร์ (Pointer) กับจำนวนของข้อมูล (The number elements) และชนิดของข้อมูลซึ่งอยู่ใน (void *message, int count, MPI_datatype datatype) นั้นเอง
- source และ dest เป็นตัวระบุอันดับของโพรเซสที่จะส่งหรือรับ ซึ่งเราสามารถใช้แบบ Wildcard ได้คือการใช้ MPI_ANY_SOURCE จะใช้ในกรณีที่ผู้รับพร้อมที่จะรับข้อความจากใครก็ได้

tag เป็นตัวระบุถึงตัวข้อความระหว่างผู้ส่งและผู้รับ ซึ่งสามารถใช้ MPI_ANY_TAG ได้ ซึ่งมันจะมีประโยชน์มากเมื่อลำดับของข้อความที่สำคัญ